Data:

```
store> db.products.find()
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  },
  {
    _id: 3,
    name: 'Keyboard',
    category: 'Electronics',
    price: 75,
    quantity: 30,
    tags: [ 'mechanical', 'rgb' ]
  },
  {
    _id: 4,
    name: 'T-shirt',
    category: 'Clothing',
    price: 20,
    quantity: 100,
    tags: [ 'cotton', 'casual' ]
  },
  {
    _id: 5,
    name: 'Jeans',
    category: 'Clothing',
    price: 60,
    quantity: 75,
    tags: [ 'denim', 'casual' ]
  {
    _id: 6,
    name: 'Book',
    category: 'Books',
    price: 15,
    quantity: 200,
    tags: [ 'fiction', 'novel' ]
  },
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 8,
    name: 'Backpack',
    category: 'Accessories',
    price: 40,
    quantity: 60,
    tags: [ 'durable', 'waterproof' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  },
  {
    _id: 10,
    name: 'Sweater',
    category: 'Clothing',
    price: 45,
    quantity: 80,
    tags: [ 'wool', 'warm' ]
  }
]
```

Step 1: Using Comparison Operators - $eq, $gt, $lt, $gte, $lte, $ne

1. $eq

```
store> db.products.find({ price: { $eq: 25 } })
[
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  }
]
```

Explanation: Taking a product with a fixed price of 25. Result: Mouse

2. $gt

```
store> db.products.find({ price: { $gt: 100 } })
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  }
]
```

Explanation: Taking products with price more than 100. Results: Laptop, Headphones, Smartphone

3. $it

```
store> db.products.find({ price: { $lt: 50 } })
[
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  },
  {
    _id: 4,
    name: 'T-shirt',
    category: 'Clothing',
    price: 20,
    quantity: 100,
    tags: [ 'cotton', 'casual' ]
  },
  {
    _id: 6,
    name: 'Book',
    category: 'Books',
    price: 15,
    quantity: 200,
    tags: [ 'fiction', 'novel' ]
  },
  {
    _id: 8,
    name: 'Backpack',
    category: 'Accessories',
    price: 40,
    quantity: 60,
    tags: [ 'durable', 'waterproof' ]
  },
  {
    _id: 10,
    name: 'Sweater',
    category: 'Clothing',
    price: 45,
```

Explanation: Taking products with a price less than 50

4. $gte

```
store> db.products.find({ price: { $gte: 75 } })
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 3,
    name: 'Keyboard',
    category: 'Electronics',
    price: 75,
    quantity: 30,
    tags: [ 'mechanical', 'rgb' ]
  },
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  }
]
```

Explanation: Price ≥ 75

5. $lte

```
store> db.products.find({ price: { $lte: 20 } })
[
  {
    _id: 4,
    name: 'T-shirt',
    category: 'Clothing',
    price: 20,
    quantity: 100,
    tags: [ 'cotton', 'casual' ]
  },
  {
    _id: 6,
    name: 'Book',
    category: 'Books',
    price: 15,
    quantity: 200,
    tags: [ 'fiction', 'novel' ]
  }
]
```

Explanation: Price ≤ 20

6. $ne

```
store> db.products.find()
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  },
  {
    _id: 3,
    name: 'Keyboard',
    category: 'Electronics',
    price: 75,
    quantity: 30,
    tags: [ 'mechanical', 'rgb' ]
  },
  {
    _id: 4,
    name: 'T-shirt',
    category: 'Clothing',
    price: 20,
    quantity: 100,
    tags: [ 'cotton', 'casual' ]
  },
  {
    _id: 5,
    name: 'Jeans',
    category: 'Clothing',
    price: 60,
    quantity: 75,
    tags: [ 'denim', 'casual' ]
  {
    _id: 6,
    name: 'Book',
    category: 'Books',
    price: 15,
    quantity: 200,
    tags: [ 'fiction', 'novel' ]
  },
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 8,
    name: 'Backpack',
    category: 'Accessories',
    price: 40,
    quantity: 60,
    tags: [ 'durable', 'waterproof' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  },
  {
    _id: 10,
    name: 'Sweater',
    category: 'Clothing',
    price: 45,
    quantity: 80,
    tags: [ 'wool', 'warm' ]
  }
]
```

Explanation: Price is not equal to 40

Step 2: $in dan $nin

1. $in

```
store> db.products.find({ category: { $in: ["Electronics", "Clothing"] } })
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  },
  {
    _id: 3,
    name: 'Keyboard',
    category: 'Electronics',
    price: 75,
    quantity: 30,
    tags: [ 'mechanical', 'rgb' ]
  },
  {
    _id: 4,
    name: 'T-shirt',
    category: 'Clothing',
    price: 20,
    quantity: 100,
    tags: [ 'cotton', 'casual' ]
  },
  {
    _id: 5,
    name: 'Jeans',
    category: 'Clothing',
    price: 60,
    quantity: 75,
    tags: [ 'denim', 'casual' ]
  },
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  },
  {
    _id: 10,
    name: 'Sweater',
    category: 'Clothing',
    price: 45,
    quantity: 80,
    tags: [ 'wool', 'warm' ]
  }
]
```

Explanation: Taking the Electronics and Clothing categories

2. $nin

```
store> db.products.find({ category: { $nin: ["Electronics", "Clothing"] } })
[
  {
    _id: 6,
    name: 'Book',
    category: 'Books',
    price: 15,
    quantity: 200,
    tags: [ 'fiction', 'novel' ]
  },
  {
    _id: 8,
    name: 'Backpack',
    category: 'Accessories',
    price: 40,
    quantity: 60,
    tags: [ 'durable', 'waterproof' ]
  }
]
```

Explanation: Taking other than the category

Step 3: Using Logical Operators - $and, $or, $not, $nor

1. $and

```
store> db.products.find({
|   $and: [{ category: "Electronics" }, { price: { $gt: 100 } }]
| })
|
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  }
]
```

Explanation: Electronics AND price > 100

2. $or

```
store> db.products.find({
|   $or: [{ category: "Books" }, { quantity: { $lt: 30 } }]
| })
|
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 6,
    name: 'Book',
    category: 'Books',
    price: 15,
    quantity: 200,
    tags: [ 'fiction', 'novel' ]
  },
```

```
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  }
]
```

Explanation: Books OR quantity < 30

3. $not

```
store> db.products.find({ category: { $not: { $eq: "Clothing" } } })
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  },
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  },
  {
    _id: 3,
    name: 'Keyboard',
    category: 'Electronics',
    price: 75,
    quantity: 30,
    tags: [ 'mechanical', 'rgb' ]
  },
  {
    _id: 6,
    name: 'Book',
    category: 'Books',
    price: 15,
    quantity: 200,
    tags: [ 'fiction', 'novel' ]
  },
```

```
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 8,
    name: 'Backpack',
    category: 'Accessories',
    price: 40,
    quantity: 60,
    tags: [ 'durable', 'waterproof' ]
  },
  {
    _id: 9,
    name: 'Smartphone',
    category: 'Electronics',
    price: 900,
    quantity: 15,
    tags: [ '5G', 'android' ]
  }
]
```

Explanation: Not a Clothing category

4. $nor

```
store> db.products.find({
|   $nor: [{ category: "Books" }, { price: { $gt: 100 } }]
| })
|
[
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  },
  {
    _id: 3,
    name: 'Keyboard',
    category: 'Electronics',
    price: 75,
    quantity: 30,
    tags: [ 'mechanical', 'rgb' ]
  },
  {
    _id: 4,
    name: 'T-shirt',
    category: 'Clothing',
    price: 20,
    quantity: 100,
    tags: [ 'cotton', 'casual' ]
  },
```

```
  {
    _id: 5,
    name: 'Jeans',
    category: 'Clothing',
    price: 60,
    quantity: 75,
    tags: [ 'denim', 'casual' ]
  },
  {
    _id: 8,
    name: 'Backpack',
    category: 'Accessories',
    price: 40,
    quantity: 60,
    tags: [ 'durable', 'waterproof' ]
  },
  {
    _id: 10,
    name: 'Sweater',
    category: 'Clothing',
    price: 45,
    quantity: 80,
    tags: [ 'wool', 'warm' ]
  }
]
```

Explanation: Not Books AND not price > 100

Step 4: Using Array Operators - $all, $elemMatch, $size

1. $ all

```
store> db.products.find({ tags: { $all: ["new", "powerful"] } })
[
  {
    _id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 1200,
    quantity: 5,
    tags: [ 'new', 'powerful' ]
  }
]
```

Explanation: The tag must contain both values. Result: Laptop

2. $ elemMatch

```
store> db.products.find({ tags: { $elemMatch: { $regex: "^w" } } })
[
  {
    _id: 2,
    name: 'Mouse',
    category: 'Electronics',
    price: 25,
    quantity: 50,
    tags: [ 'wired', 'ergonomic' ]
  },
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  },
  {
    _id: 8,
    name: 'Backpack',
    category: 'Accessories',
    price: 40,
    quantity: 60,
    tags: [ 'durable', 'waterproof' ]
  },
  {
    _id: 10,
    name: 'Sweater',
    category: 'Clothing',
    price: 45,
    quantity: 80,
    tags: [ 'wool', 'warm' ]
  }
]
```

Explanation: At least one tag begins with the letter "w"

3. $ size

```
store> db.products.find({ tags: { $size: 1 } })
```

Explanation: Product with 1 tag. Result: None

Step 5: Combining Operators

1. Example 1

```
store> db.products.find({
|   $and: [
|     { category: "Electronics" },
|     { price: { $gt: 50 } },
|     { tags: "wireless" }
|   ]
| })
|
[
  {
    _id: 7,
    name: 'Headphones',
    category: 'Electronics',
    price: 150,
    quantity: 20,
    tags: [ 'wireless', 'noise-cancelling' ]
  }
]
```

Explanation: Electronics + price > 50 + tag wireless. Result: Headphones

2. Example 2

```
store> db.products.find({
|   $or: [
|     {
|       $and: [
|         { category: "Clothing" },
|         { price: { $lt: 50 } }
|       ]
|     },
|     { tags: { $size: 1 } }
|   ]
| })
|
[
  {
    _id: 4,
    name: 'T-shirt',
    category: 'Clothing',
    price: 20,
    quantity: 100,
    tags: [ 'cotton', 'casual' ]
  },
  {
    _id: 10,
    name: 'Sweater',
    category: 'Clothing',
    price: 45,
    quantity: 80,
    tags: [ 'wool', 'warm' ]
  }
]
```

Explanation: Clothing price < 50 OR tag = 1

Validation Results

- All code runs as intended

- When changing parameters (name or category), the results change according to the code logic

Troubleshooting

Some problems encountered:

1. **Syntax error**
    - Missing {}, [], or "" symbols
    - Solution: Recheck the syntax
2. **Data not appearing.**
    - Cause: Incorrect condition
    - Solution: Check the data content with find() or recheck the code
3. **Case sensitivity**
    - MongoDB is case sensitive
    - Solution: Use regular expressions if necessary